

MOTOR BD v1.0

Motor de Base de Datos con Árbol AVL Autobalanceado

Documentación Técnica — Proyecto Final

Integrantes: Anyelo Esteban Casas Zapata (Código: 20251020106)
Diego Alejandro Yáñez Zabala (Código: 20251020103)
Wilfer Arbey Herrera Garzón (Código: 20251020071)

Asignatura: Ciencias de la Computación I — Grupo 020-83 — Semestre 2026-I

Docente: Ing. Simar Enrique Herrera Jiménez

Institución: Universidad Distrital Francisco José de Caldas

1. Diseño Arquitectónico

El motor de base de datos se encuentra estructurado en cinco módulos independientes. El flujo de dependencias es estrictamente unidireccional para mitigar el acoplamiento cíclico de componentes. El módulo encargado del árbol AVL opera como una estructura abstracta pura, aislando por completo las reglas de negocio, los catálogos y los mecanismos de persistencia en almacenamiento secundario.

1.1. Componentes del Sistema

Índice / Árbol (`ArbolAVL`, `NodoAVL`): Provee el soporte para el árbol genérico balanceado. Resuelve las operaciones de inserción, búsqueda puntual, eliminación, consultas por rangos y funciones de volcado en formato ASCII.

Catálogo (`Catalogo`, `Esquema`, `Campo`, `TipoDato`): Administra y valida la integridad estructural de las tablas del sistema. Soporta la definición tanto de esquemas fijos como de esquemas libres, verificando tipos de datos y restricciones de unicidad sobre la clave primaria.

Almacenamiento (`Registro`, `Espacio`, `GestorEspacios`): Coordina las transacciones CRUD en memoria volátil. Cada instancia de la clase `Espacio` encapsula su propio árbol indexado mediante una llave primaria (*PK*).

Persistencia (`GestorPersistencia`): Maneja el flujo de I/O en disco. Traduce el estado de la memoria a archivos estructurados independientes (`.schema` en formato JSON y `.json` en formato JSON Lines).

Parser / CLI (`Motor`, `ResultadoComando`, `REPL`): Analiza de forma léxica y sintáctica las cadenas de texto del usuario. Implementa el ciclo interactivo de lectura y el procesamiento batch mediante scripts de comandos.

1.2. Flujo de una Operación

El procesamiento de una instrucción sigue una secuencia lineal: el componente `REPL` captura la entrada de texto; el método `Motor.ejecutar()` realiza el parseo e invoca la operación correspondiente en el componente `Espacio`; este delega el ordenamiento algorítmico al `ArbolAVL` y, al consolidarse la mutación, el objeto `GestorPersistencia.guardar()` actualiza los ficheros en disco de forma síncrona.

1.3. Estructura de Paquetes

```
motor/  
  arbol/          --> NodoAVL.java, ArbolAVL.java  
  catalogo/       --> Campo.java, TipoDato.java, Esquema.java, Catalogo.java  
  almacenamiento/ --> Registro.java, Espacio.java, GestorEspacios.java  
  persistencia/   --> GestorPersistencia.java  
  parser/         --> Motor.java, ResultadoComando.java  
  cli/            --> REPL.java  
  test/           --> TestMotor.java
```

2. Especificación del Lenguaje de Comandos

El motor interpreta un subconjunto simplificado del lenguaje SQL convencional. El analizador sintáctico opera de forma *case-insensitive*. Las instrucciones se delimitan opcionalmente mediante el carácter de punto y coma (;). El búfer del REPL concatena múltiples líneas de entrada hasta detectar el delimitador de cierre, permitiendo construcciones lógicas formateadas en varios bloques. Los comentarios se introducen mediante el prefijo de doble guion (--).

2.1. Gestión de Espacios (DDL)

- `CREATE SPACE nombre (campo tipo [PK], ...)`: Inicializa una tabla relacional bajo un esquema rígido.
- `CREATE SPACE nombre`: Inicializa un espacio no relacional orientado a esquemas dinámicos o libres.
- `DROP SPACE nombre`: Libera la memoria ocupada por la tabla y destruye sus archivos físicos del disco.
- `SHOW SPACES`: Interroga al catálogo del sistema para listar las tablas cargadas.
- `DESC nombre`: Renderiza los metadatos de la tabla incluyendo tipos y campos clave.

2.2. Tipos de Datos Soportados y Mapeo en Java

- **ENTERO** (Alias admisibles: `INT`, `INTEGER`) → Mapeado internamente a `java.lang.Integer`.
- **TEXTO** (Alias admisibles: `TEXT`, `STRING`) → Mapeado internamente a `java.lang.String`.
- **REAL** (Alias admisibles: `FLOAT`, `DOUBLE`) → Mapeado internamente a `java.lang.Double`.
- **BOOLEAN** (Alias admisible: `BOOL`) → Mapeado internamente a `java.lang.Boolean`.

2.3. Operaciones de Manipulación de Datos (DML)

- `INSERT INTO nombre VALUES (val1, val2, ...)`
- `INSERT INTO nombre (c1, c2) VALUES (val1, val2)`
- `SELECT * FROM nombre`
- `SELECT * FROM nombre WHERE campo op valor`

- `SELECT * FROM nombre WHERE campo BETWEEN val1 AND val2`
- `UPDATE nombre SET c1=val1, c2=val2 WHERE campo op valor`
- `DELETE FROM nombre WHERE campo op valor`
- `DELETE FROM nombre ALL`

Operadores condicionales evaluados por el lexer: =, !=, <>, <, <=, >, >=.

2.4. Comandos Auxiliares del Sistema

- `TREE nombre`: Vuelca la topología binaria del árbol AVL en la consola (alturas y factores de balance).
- `SAVE nombre`: Sincroniza explícitamente el espacio referenciado volcando los datos a disco.
- `SAVE ALL`: Fuerza la serialización total del catálogo actual en los ficheros locales correspondientes.
- `EXIT / QUIT`: Termina el ciclo infinito del REPL y realiza un cierre limpio del hilo del motor.

3. Diseño del Árbol AVL

La estructura de indexación se desarrolló desde cero sin utilizar colecciones de `java.util`. El componente se modeló como `ArbolAVL<K extends Comparable<K>, V>`, asegurando compatibilidad con cualquier clave primaria con relación de orden total. Las claves alfanuméricas se procesan en minúsculas para estandarizar las búsquedas. Cada clase `NodoAVL` almacena de forma explícita su clave, valor asignado, enlaces hacia los nodos hijos y su altura calculada.

3.1. Invariantes Estructurales del AVL

- **Propiedad de Búsqueda Binaria (BST)**: Para cada nodo raíz n , se cumple estrictamente la relación:

$$\text{clave}(\text{izq}) < \text{clave}(n) < \text{clave}(\text{der})$$

3.2. Mecanismos de Rebalanceo Mecánico

Si una inserción o remoción altera los invariantes, la rutina `balancear(n)` intercepta el retorno de la recursión en los nodos ancestros. Evalúa cuatro tipos de desbalances aplicando rotaciones en tiempo constante $\mathcal{O}(1)$:

- **Caso LL (Izquierda-Izquierda)**: Si $\text{fb}(n) > 1$ y $\text{fb}(n.\text{izq}) \geq 0$, se ejecuta una rotación simple a la derecha mediante `rotDer(n)`.
- **Caso LR (Izquierda-Derecha)**: Si $\text{fb}(n) > 1$ y $\text{fb}(n.\text{izq}) < 0$, se ejecuta una combinación doble consistente en `rotIzq(n.izq)` seguida de `rotDer(n)`.
- **Caso RR (Derecha-Derecha)**: Si $\text{fb}(n) < -1$ y $\text{fb}(n.\text{der}) \leq 0$, se mitiga mediante una rotación simple a la izquierda utilizando `rotIzq(n)`.
- **Caso RL (Derecha-Izquierda)**: Si $\text{fb}(n) < -1$ y $\text{fb}(n.\text{der}) > 0$, se rebalancea mediante `rotDer(n.der)` seguido de la rotación complementaria `rotIzq(n)`.

3.3. Algoritmo de Eliminación y Consultas Especializadas

La remoción de nodos internos provistos de dos subárboles se solventa localizando el **sucesor inorden** (el elemento mínimo del subárbol derecho). Las propiedades lógicas de dicho nodo se transfieren a la posición actual, destruyendo posteriormente la hoja duplicada en el extremo inferior del árbol para luego recalcular los factores de equilibrio.

El método `rango(desde, hasta)` efectúa una exploración inorden acotada mediante técnicas de poda binaria. La exploración del subárbol izquierdo se omite si la clave del nodo actual es menor o igual al límite `desde`. Análogamente, se restringe el descenso por la rama derecha si la clave supera el parámetro `hasta`. Este comportamiento reduce el costo computacional a un orden de $\mathcal{O}(\log n + k)$.

4. Formato de Persistencia y Política de Recuperación

4.1. Estructura Física en Disco

La persistencia de la información se gestiona de manera local dentro de directorios estructurados:

- **Metadatos (`data/esquemas/<tabla>.schema`):** Almacena un único renglón en texto JSON que define las características estructurales de la tabla, detallando el nombre, naturaleza del esquema, campo indexado clave y tipos de datos admitidos.
- **Registros (`data/espacios/<tabla>.json`):** Sigue la especificación **JSON Lines**. Cada fila lógica de la tabla se codifica como un objeto JSON compacto e independiente por cada salto de línea del archivo de texto.

4.2. Estrategia de Sincronización y Recuperación

El motor aplica una política de persistencia determinista y síncrona. Cualquier cambio de estado ejecutado a través de comandos `INSERT`, `UPDATE` o `DELETE` desencadena la sobrescritura atómica del archivo de texto asociado utilizando canales de tipo `FileWriter(f, false)`, impidiendo la mezcla accidental con bloques remanentes corruptos.

Durante el arranque de la aplicación, el método `GestorPersistencia.cargarTodo()` examina el directorio de metadatos, instancia los esquemas lógicos en el catálogo y decodifica secuencialmente el archivo `.json` fila por fila. Cada registro es cargado en el índice en un tiempo total estimado de $\mathcal{O}(n \log n)$. Ante registros inválidos, el analizador omite la línea enviando una notificación descriptiva al canal `stderr`, impidiendo fallas catastróficas en el resto de los componentes del catálogo.

5. Análisis de Complejidad Temporal y Espacial

A continuación se detallan los órdenes de complejidad computacional que rigen los componentes algorítmicos internos del motor:

- Inserción por clave primaria (PK):** Complejidad de $\mathcal{O}(\log n)$. El algoritmo efectúa un descenso binario puro acotado por la altura máxima teórica del árbol AVL, la cual cumple la condición estricta $h \leq 1,44 \log_2(n + 2)$.
- Búsqueda puntual por PK:** Complejidad de $\mathcal{O}(\log n)$. Realiza una exploración descendente a través de los nodos del índice sin incurrir en procesos de backtracking.
- Búsqueda por rango (PK):** Complejidad de $\mathcal{O}(\log n + k)$, donde k representa el número de registros en el intervalo. La optimización mediante poda binaria evita examinar subárboles fuera de los límites.

Eliminación por PK: Complejidad de $\mathcal{O}(\log n)$. Involucra la localización de la clave, la extracción opcional del nodo sucesor inorden y la propagación del balanceo en los nodos padres.

UPDATE / DELETE por campo no indexado: Complejidad de $\mathcal{O}(n)$. Al carecer de índices secundarios implementados, el motor realiza un escaneo lineal completo recorriendo el árbol en inorden.

Rotaciones de balanceo: Complejidad de $\mathcal{O}(1)$. Corresponde únicamente a intercambios locales de referencias de memoria y reasignación matemática de alturas en dos nodos.

Carga inicial del sistema: Complejidad de $\mathcal{O}(n \log n)$. Requiere procesar un número n de inserciones individuales consecutivas desde los archivos físicos.

Consumo de memoria volátil: Complejidad de $\mathcal{O}(n)$. Escala linealmente en función de los datos almacenados; cada registro se aloja exactamente en un objeto de tipo `NodoAVL`.

6. Suite de Pruebas y Datasets

6.1. Suite de Pruebas Interna (`TestMotor.java`)

Las pruebas funcionales se concentran en la clase ejecutable `src/test/TestMotor.java`. Su diseño prescinde de librerías de terceros (como JUnit) para garantizar la portabilidad directa desde entornos de consola pura. El módulo inicializa un directorio de trabajo exclusivo denominado `data_test/`, el cual se limpia y destruye automáticamente al concluir los experimentos para asegurar la idempotencia del framework.

La suite valida de forma automática los siguientes 10 escenarios clave del motor:

1. **Gestión de esquemas:** Creación de espacios lógicos relacionales y libres mediante sentencias `CREATE`, uso de `SHOW SPACES` y control de excepciones ante tablas duplicadas.
2. **Operaciones INSERT:** Inserciones con tipos de datos correctos, rechazo de claves duplicadas en el árbol AVL y procesamiento de campos variables en esquemas dinámicos.
3. **Consultas SELECT puntuales:** Evaluación de filtros relacionales con cláusulas `WHERE` sobre tipos primitivos (enteros, reales, cadenas de caracteres y booleanos).
4. **Consultas SELECT con rango:** Comprobación de límites lógicos mediante operadores `BETWEEN` considerando conjuntos de datos parciales, totales o vacíos.
5. **Modificación de registros (UPDATE):** Validación del correcto reemplazo de datos en memoria indexados tanto por clave primaria como por atributos secundarios.
6. **Remoción de datos (DELETE):** Control de la disminución del tamaño del árbol tras ejecuciones de limpieza puntual o vaciado masivo de registros con `DELETE ALL`.
7. **Monitoreo del Índice (TREE):** Verificación de la integridad topológica del árbol balanceado mediante volcados de texto ASCII con factores de balance vigentes.
8. **Estructura del Catálogo (DESC):** Inspección detallada del formato de las tablas cargadas en el sistema (campos, tipos, claves primarias, altura y cantidad de elementos).
9. **Persistencia y Persistencia Inversa:** Ejecución de comandos `SAVE ALL`, cierre controlado del proceso y validación de la posterior reconstrucción de datos instanciando un nuevo proceso del motor.
10. **Eliminación de tablas (DROP SPACE):** Remoción física de los esquemas y control de excepciones al intentar interrogar espacios lógicos inexistentes.

6.2. Datasets Incorporados

El entorno provee dos scripts de automatización ubicados en la carpeta local de datos para su invocación batch:

- **script_pequeno.sql**: Configura una tabla relacional estructurada con 5 registros (id ENTERO PK, nombre TEXTO, promedio REAL, semestre ENTERO, activo BOOLEAN). Evalúa la respuesta de los componentes INSERT, SELECT BETWEEN, TREE y la sincronización a disco.
- **script_mediano.sql**: Carga una colección extendida de 20 filas operando sobre el esquema productos (codigo TEXTO PK, nombre TEXTO, precio REAL, stock ENTERO, categoria TEXTO). Comprueba búsquedas complejas por rangos alfanuméricos, modificaciones condicionales y comandos de descripción de catálogo.

Para inicializar y reproducir estos datasets desde consola se utilizan los comandos:

```
java -cp build/classes cli.REPL --script data/script_pequeno.sql
java -cp build/classes cli.REPL --script data/script_mediano.sql
```